Polimorfizm i Dziedziczenie w Programowaniu Obiektowym

Barbara Orpel

18.11.2024

Spis treści:

1. Wprowadzenie
2. Polimorfizm klas
3. Polimorfizm funkcji
4. Typy dziedziczenia i widoczność składników klas
5. Funkcje wirtualne
6. Tworzenie i niszczenie obiektów klas bazowych i dziedziczących
7. Podsumowanie
8. Bibliografia

1. Wprowadzenie

Programowanie obiektowe (OOP) to paradygmat programowania, który koncentruje się na obiektach jako podstawowych jednostkach struktury kodu. W OOP program jest organizowany wokół obiektów, które reprezentują elementy rzeczywistości, a ich interakcje są modelowane za pomocą metod i atrybutów. Podejście to umożliwia łatwe rozszerzanie, modyfikowanie oraz ponowne wykorzystywanie kodu, co sprzyja tworzeniu bardziej złożonych i skalowalnych aplikacji.

Podstawowe pojęcia programowania obiektowego:

- **Klasa** – Definicja obiektu, która określa jego właściwości (atrybuty) i działania (metody).
- **Obiekt** – Konkretna instancja klasy, reprezentująca pojedynczy egzemplarz tej definicji.
- **Dziedziczenie** – Mechanizm, który pozwala tworzyć nowe klasy na podstawie już istniejących, co sprzyja ponownemu wykorzystaniu kodu.
- **Polimorfizm** – Zdolność obiektów różnych klas do bycia traktowanymi w jednolity sposób dzięki wspólnym metodom.
- **Hermetyzacja** – Ukrywanie wewnętrznej implementacji obiektu i zapewnianie dostępu do niego tylko poprzez zdefiniowany interfejs publiczny.
- **Konstruktor** – Specjalna metoda w klasie, która jest automatycznie wywoływana w momencie tworzenia obiektu tej klasy. Konstruktor jest odpowiedzialny za inicjalizację obiektu, czyli przypisanie początkowych wartości do jego atrybutów i przygotowanie go do użycia. Na przykład w C++ konstruktor nosi tę samą nazwę co klasa.
- **Destruktor** – Specjalna metoda wywoływana automatycznie, gdy obiekt jest niszczony lub przestaje być używany. Destruktor pozwala na zwolnienie zasobów zajmowanych przez obiekt (takich jak pamięć lub uchwyty do plików), co jest szczególnie ważne w językach, które wymagają ręcznego zarządzania zasobami, takich jak C++. W C++ destruktor nosi nazwę klasy poprzedzoną symbolem ~

2. Polimorfizm klas

Polimorfizm to zdolność obiektu do przyjmowania różnych form, co oznacza, że ten sam kod może być używany do różnych typów danych. W kontekście klas polimorfizm umożliwia korzystanie z jednej klasy bazowej, która jest wspólna dla różnych klas pochodnych. Dzięki temu obiekty różnych typów mogą być traktowane w sposób uniwersalny.

Przykład:

Załóżmy, że mamy klasę „Bazowa” oraz klasy pochodne „PochodnaA” i „PochodnaB”. Dzięki polimorfizmowi możemy stworzyć metodę „akcja”, która działa dla wszystkich typów obiektów dziedziczących po „Bazowa”.

```
class Bazowa {
public:
    virtual void akcja() {
        std::cout << "Akcja w klasie Bazowa" << std::endl;
    }
};
class PochodnaA : public Bazowa {
public:
    void akcja() override {
        std::cout << "Akcja w klasie PochodnaA" << std::endl;
    }
};
class PochodnaB : public Bazowa {
public:
    void akcja() override {
        std::cout << "Akcja w klasie PochodnaB" << std::endl;
    }
};
int main() {
    std::vector<Bazowa*> obiekty = { new PochodnaA(), new PochodnaB() };
    for (Bazowa* obiekt : obiekty) {
        obiekt->akcja();
    }
    for (Bazowa* obiekt : obiekty) {
        delete obiekt;
    }
    return 0;
}
```

W powyższym przykładzie, klasa Bazowa definiuje metodę akcja, którą następnie nadpisują klasy pochodne PochodnaA oraz PochodnaB. Dzięki polimorfizmowi możemy umieścić wskaźniki do obiektów klas PochodnaA i PochodnaB w tej samej kolekcji obiektów i wywołać metodę akcja() na każdym z nich. C++ dynamicznie wywołuje odpowiednią wersję metody akcja, zależnie od rzeczywistego typu obiektu, dzięki słowu kluczowemu virtual.

3. Polimorfizm funkcji

Polimorfizm funkcji, nazywany również przeciążeniem, umożliwia definiowanie kilku wersji tej samej funkcji, które różnią się typami lub liczbą parametrów.
W C++ możemy przeciążać funkcje, tworząc kilka wersji funkcji o tej samej nazwie, ale z różnymi parametrami.

Przykład:

Poniżej przedstawiam klasę „Przykladowa”, która posiada dwie wersje metody „metoda” o różnej liczbie parametrów.

```
class Przykladowa {
public:
    void metoda() {
        std::cout << "Metoda bez parametrów" << std::endl;
    }

    void metoda(int liczba) {
        std::cout << "Metoda z jednym parametrem: " << liczba << std::endl;
    }
};

int main() {
    Przykladowa przyklad;
    przyklad.metoda();
    przyklad.metoda(10);
    return 0;
}
```

W przykładzie powyżej metoda „metoda” została przeciążona w klasie „Przykladowa”. Pierwsza wersja metoda() nie przyjmuje żadnych parametrów, natomiast druga wersja metoda(int liczba) przyjmuje jeden parametr typu int. Dzięki temu przeciążeniu możemy wywołać odpowiednią wersję metody w zależności od liczby argumentów. Jest to przykład polimorfizmu na poziomie funkcji, umożliwiający różne zachowania dla tej samej nazwy funkcji.

4. Typy dziedziczenia i widoczność składników klas

Dziedziczenie umożliwia tworzenie nowych klas na podstawie już istniejących, co pozwala na ponowne użycie kodu oraz jego rozszerzenie. Wyróżnia się kilka typów dziedziczenia:

- **Dziedziczenie pojedyncze** - klasa dziedziczy bezpośrednio od jednej klasy bazowej.
- **Dziedziczenie wielokrotne** - klasa może dziedziczyć po więcej niż jednej klasie bazowej.
- **Dziedziczenie wielopoziomowe** - dziedziczenie odbywa się przez kilka poziomów (klasa pochodna dziedziczy po innej klasie pochodnej).

Widoczność składników klas odnosi się do poziomu dostępu do pól i metod w klasach:

- **Prywatne** (private) – dostępne tylko wewnątrz klasy.
- **Chronione** (protected) – dostępne w klasie oraz klasach pochodnych.
- **Publiczne** (public) – dostępne dla wszystkich.

Przykład:

W poniższym przykładzie Bazowa zawiera prywatne, chronione i publiczne pola.

```
class Bazowa {
private:
    std::string prywatnePole = "Prywatne";
protected:
    std::string chronionePole = "Chronione";
public:
    std::string publicznePole = "Publiczne";

    void pokazPola() {
        std::cout << prywatnePole << ", " << chronionePole << ", " << publicznePole << std::endl;
    }
};

class Pochodna : public Bazowa {
public:
    void pokazChronione() {
        std::cout << "Dostęp do chronionego: " << chronionePole << std::endl;
    }
};

int main() {
    Pochodna obiekt;
    obiekt.pokazChronione();
    obiekt.pokazPola();
    return 0;
}
```

W przykładzie powyżej mamy klasę Bazowa, która definiuje trzy pola: prywatne (prywatnePole), chronione (chronionePole) i publiczne (publicznePole). Pole prywatnePole jest dostępne tylko w obrębie klasy Bazowa i nie można go bezpośrednio uzyskać w klasie Pochodna. chronionePole jest dostępne w klasie Pochodna i może być bezpośrednio użyte w metodzie tej klasy. publicznePole jest dostępne wszędzie. Metoda pokazPola() w klasie Bazowa umożliwia dostęp do prywatnePole nawet z klasy Pochodna, ale wyłącznie poprzez wywołanie tej metody.

5. Funkcje wirtualne

Funkcje wirtualne to funkcje, które mogą być nadpisywane w klasach pochodnych, umożliwiając dynamiczne wywoływanie odpowiednich wersji funkcji w zależności od typu obiektu, który wywołuje daną metodę. W C++ funkcję należy oznaczyć słowem kluczowym virtual w deklaracji w klasie bazowej, aby kompilator rozumiał, że metoda może być nadpisana w klasach pochodnych. Wówczas wywołanie metody na wskaźniku lub referencji do klasy bazowej uruchomi odpowiednią wersję metody, zależnie od typu obiektu.

Przykład:

W C++ konieczne jest użycie słowa virtual przy metodzie „działanie” w klasie „Bazowa”, aby umożliwić dynamiczne wywoływanie odpowiedniej wersji metody.

```
class Bazowa {
public:
    virtual void dzialanie() {
        std::cout << "Działanie w klasie Bazowa" << std::endl;
    }
};

class Pochodna : public Bazowa {
public:
    void dzialanie() override {
        std::cout << "Działanie w klasie Pochodna" << std::endl;
    }
};

int main() {
    Bazowa* obiektBazowy = new Bazowa();
    Bazowa* obiektPochodny = new Pochodna();

    obiektBazowy->dzialanie();
    obiektPochodny->dzialanie();

    delete obiektBazowy;
    delete obiektPochodny;
    return 0;
}
```

W przykładzie powyżej metoda „działanie” w klasie „Bazowa” jest oznaczona jako virtual, co oznacza, że może być nadpisana przez klasy pochodne. Dzięki temu, kiedy dzialanie() jest wywoływane na wskaźniku obiektPochodny typu Bazowa*, wywoływana jest wersja z klasy „Pochodna”, ponieważ obiekt obiektPochodny jest instancją tej klasy. Słowo override przy metodzie „działanie” w klasie „Pochodna” pomaga kompilatorowi sprawdzić, że metoda faktycznie nadpisuje wersję z klasy bazowej

6. Tworzenie i niszczenie obiektów klas bazowych i dziedziczących

Konstruktor klasy bazowej jest wywoływany przed konstruktorem klasy pochodnej, a destruktor klasy pochodnej jest wywoływany przed destruktorem klasy bazowej, co pozwala precyzyjnie kontrolować tworzenie i niszczenie obiektów.

Przykład:

Poniżej pokazano tworzenie i niszczenie obiektów w C++.

```
class Bazowa {
public:
    Bazowa() {
        std::cout << "Tworzenie obiektu klasy Bazowa\n";
    }
    virtual ~Bazowa() {
        std::cout << "Niszczenie obiektu klasy Bazowa\n";
    }
};

class Pochodna : public Bazowa {
public:
    Pochodna() {
        std::cout << "Tworzenie obiektu klasy Pochodna\n";
    }
    ~Pochodna() {
        std::cout << "Niszczenie obiektu klasy Pochodna\n";
    }
};

int main() {
    Bazowa* obiekt = new Pochodna();
    delete obiekt;
    return 0;
}
```

W powyższym kodzie konstruktor Bazowa jest wywoływany przed konstruktorem Pochodna. Podobnie, destruktor Pochodna jest wywoływany przed destruktorem Bazowa, dzięki czemu zasoby alokowane przez Pochodna mogą zostać zwolnione przed zasobami Bazowa. Słowo kluczowe virtual przy destruktorze Bazowa zapewnia, że podczas wywołania delete dla obiektu typu Bazowa*, zostanie wywołany destruktor Pochodna, co jest kluczowe dla poprawnego zarządzania pamięcią w przypadku dziedziczenia.

7. Podsumowanie

Polimorfizm i dziedziczenie to fundamentalne pojęcia w programowaniu obiektowym, które umożliwiają łatwiejsze zarządzanie kodem i zwiększają jego elastyczność. Dziedziczenie pozwala na ponowne wykorzystanie istniejącego kodu, a polimorfizm umożliwia traktowanie obiektów różnych typów w sposób jednolity. Dzięki temu programowanie obiektowe jest potężnym narzędziem pozwalającym na tworzenie rozbudowanych i skalowalnych aplikacji.

8. Bibliografia

- Stroustrup, B. (2013). The C++ Programming Language (4th ed.). Addison-Wesley
- Lafore, R. (2002). *Object-Oriented Programming in C++* (4th ed.). SAMS Publishing